

Language Design Proposal: Renegade

Student Name(s): Sean Lachica, Josh Penaojas, Javier Pulido

Language Name: Renegade

Target Language: JavaScript

Language Description: A Haskell-like language featuring typeclasses, function overloading, and higher-order functions, which compiles to JavaScript.

Key Features: Typeclasses, structs, higher-order functions, function overloading, syntax not based on S-expressions.

Planned Restrictions: No generics, no algebraic data types, or any other way to make a recursive type. This dramatically simplifies things, as we will always know what method is being referred to at compile-time.

Suggested Scoring and Justification:

- **Lexer:** 10%. Only support for reserved words, identifiers, and integers. No comments.
- **Parser:** 20%. Does not use S-expressions.
- **Typechecker:** 40%. Higher-order functions, typeclasses, structs, named function overloading.
- **Code Generator:** 30%. Most of the work is expected to be in handling typeclasses and function overloading, as JavaScript doesn't support these.

Concrete Syntax:

`var` is a variable

`structname` is the name of a struct

`traitname` is the name of a trait (typeclass)

`i` is an integer

`comma_type ::= [type (`,` type)*]`

`type ::=`

``Int` | `Void` | `Boolean` | Built-in types`

``Self` | Refers to our own type in a trait`

`structname | Structs are a valid kind of type`

``(` type `)` | Parenthesized type`

``(` comma_type `)` `=>` type Higher-order function`

param ::= var `:` type

comma_param ::= [param (`,` param)*]

structdef ::= `struct` structname `{` comma_param `}`

Definition of an abstract method

abs_methoddef ::= `method` var (` comma\_param `) `:` type `;`

Definition of a concrete method

conc_methoddef ::=
`method` var (` comma\_param `) `:` type `{` stmt* `}`

Definition of a trait (typeclass)

traitdef ::= `trait` traitname `{` abs_methoddef* `}`

Definition of an implementation of a typeclass

impldef ::= `impl` traitname `for` type `{` conc_methoddef* `}`

Definition of a toplevel function

funcdef ::= `func` var (` comma\_param `) `:` type
`{` stmt* `}`

stmt ::= `let` param `=` exp `;` | **Variable declaration**
var `=` exp `;` | **Assignment**
`if` (` exp `) stmt [`else` stmt] | **if**
`while` (` exp `) stmt | **while**
`break` `;` | **break**
`println` (` exp `) | **Printing something**
`{` stmt* `}` | **Block**
`return` [exp] `;` | **Return**
exp `;` **Expression statements**

struct_actual_param ::= var `:` exp

struct_actual_params ::=
[struct_actual_param (`,` struct_actual_param)*]

primary_exp ::= i | var | **Integers and variables**
`true` | `false` | **Booleans**
`self` | **Instance on which we call a method**
(` exp `) | **Parenthesized expression**

Creates a new instance of a struct

`new` structname `{` struct_actual_params `}`

Accessing a struct field or method

```

dot_exp ::= primary_exp (`. ` var)*
call_exp ::= dot_exp (`(` comma_exp `)`)*
mult_exp ::= call_exp ((`*` | `/`) call_exp)*
add_exp ::= mult_exp ((`+` | `-`) mult_exp)*
less_than_exp ::= add_exp [<`<` add_exp]
equals_exp ::= less_than_exp [<`==` | `!=`] less_than_exp
exp ::= equals_exp

program_item ::= structdef | traitdef | impldef | funcdef
program ::= program_item* stmt* stmt* is the entry point

```

Example: Generic-looking add method

```

trait Addable {
  method add(other: Self): Self;
}

trait Printable {
  method print(): Void;
}

struct IntWrapper {
  value: Int
}

impl Addable for Int {
  method add(other: Int): Int {
    return self + other;
  }
}

impl Addable for IntWrapper {
  method add(other: IntWrapper): IntWrapper {
    return new IntWrapper { value: self.value + other.value };
  }
}

impl Printable for Int {
  method print(): Void {
    println(self);
  }
}

```

```
}
```

```
impl Printable for IntWrapper {  
  method print(): Void {  
    println(self.value);  
  }  
}
```

```
let a1: Int = 5;  
let a2: IntWrapper = new IntWrapper { value: 7 };  
let a3: Int = a1.add(2);  
let a4: IntWrapper = a2.add(new IntWrapper { value: 3 });  
a3.print();  
a4.print();
```